Iteration 2 — Change Report

One problem, one fix: obstacles must not vanish from the cost map when the sensor loses sight of them

File changed: `imprimis_navigation/config/nav2/Course2027.yaml` only. The behaviour tree is untouched, and so is everything else in the params file. The previous versions of both files are in `iterations/2/OldFiles/`.

The change in one line: the `obstacle_layer` in both cost maps is replaced by `spatio_temporal_voxel_layer/SpatioTemporalVoxelLayer`, which clears by sensor frustum and by time instead of by 2D ray-tracing. Everything added is tagged `ITER2`.

1. Why the obstacles disappeared

Two things combine, and neither is a tuning mistake — ray-trace clearing simply cannot do what we need with this sensor at this height.

a. Nav2's ray-trace clearing is 2D

`ObstacleLayer::raytraceFreespace` iterates the cloud with only two iterators:

```
sensor_msgs::PointCloud2ConstIterator<float> iter_x(cloud, "x");
sensor_msgs::PointCloud2ConstIterator<float> iter_y(cloud, "y");
```

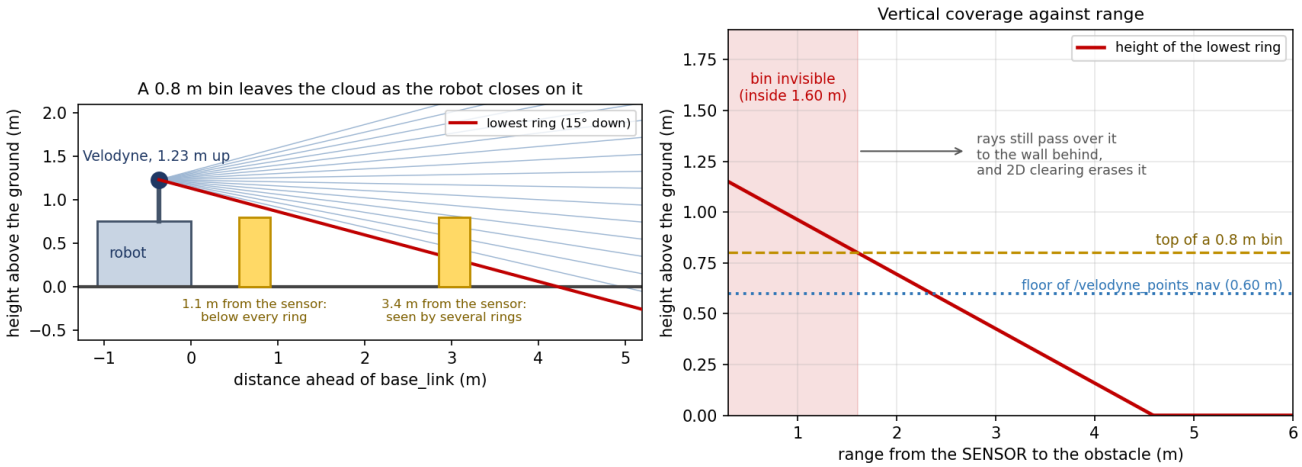
There is no `z`. Every return is flattened to the ground plane and a line is drawn from the sensor to it, setting every cell it crosses to free. A ring that passes *over* a bin and strikes the wall behind therefore erases the bin.

The good news, and it is worth stating because it is the thing that was feared: clearing is driven **only by returns that actually came back**. A direction with no return produces no ray and clears nothing, so pure occlusion and loss of field of view never erase anything. The erasure is specifically the ray-passes-over-it case.

b. The Velodyne is too high to see a bin up close

From `/tf_static` in the iteration-2 bags, the sensor sits **1.230 m** above the ground (base_link is 0.128 m up, the Velodyne 1.103 m above that) and 0.369 m behind base_link. With the VLP-16's 15° downward look, its lowest ring is at **1.23** − **0.268 × range** metres. An object whose top is at height H falls below every ring once the sensor is within **(1.23** − **H) / tan 15°**:

Top of the object	Invisible once the sensor is within	That is this far in front of the robot's nose
0.60 m	2.35 m	1.76 m
0.79 m (one measured in real_bag2)	1.64 m	1.05 m
0.90 m	1.23 m	0.64 m
1.10 m	0.49 m	reaches the robot first



Left: the 16 rings from the mast. A bin 1.1 m from the sensor is under all of them, while the same bin at 3.4 m is crossed by several. Right: the same thing as a curve. Inside the shaded band the bin produces no returns at all, yet rays to the wall behind it keep sweeping through its cells.

c. A third of the problem is upstream, and is not fixed here

`/velodyne_points_nav` — the only cloud either cost map reads — contains **nothing below 0.60 m above the ground** in essentially every frame of both bags (minimum z +0.472 in base_link, in 195 of 195 frames of real_bag1 and in the median frame of real_bag2). That is exactly `ramp_detector`'s `surface_max_height` of 0.60: in `surface_tools.classify_lidar` a return below that height is only called an obstacle if it also fails the `steep` test against its neighbouring rings, and in this hallway that test is apparently never rescuing one.

So the cost map only ever learns about the *top* of an object, which makes it drop out of view sooner than the raw sensor would require. **This is in `imprimis_perception`, not in the two files I may edit, so nothing was changed — but it is worth a look before the next run.** A bin shorter than 0.60 m would never be marked at all, at any range.

2. The fix

`SpatioTemporalVoxelLayer` (STVL 2.6.2, already installed) does not ray-trace. It keeps a sparse 3D voxel grid and forgets in two independent ways:

• **every** voxel decays on a timer (`voxel_decay`), so a stale mark goes away by itself — which is what iteration 1 needed clearing for in the first place;
• voxels **inside the sensor's frustum** that were not seen again decay faster (`decay_acceleration`), so something genuinely removed goes quickly.

The property we need is in the implementation: `SpatioTemporalVoxelGrid::ClearFrustums` only touches voxels for which the frustum's `IsInside()` returns true, and for `model_type: 1` that frustum is `ThreeDimensionalLidarFrustum(vertical_fov, padding, horizontal_fov, min_range, max_range)` anchored at the sensor. **A voxel the Velodyne physically cannot see is outside the frustum and is never cleared.** That is precisely the rule asked for.

The settings, and why each one

Parameter	Value	Why
`voxel_decay` / `decay_model`	15.0 / 0 (linear)	How long an **unseen** voxel survives. This is the only thing that forgets an obstacle out of the sensor's view, so it is deliberately generous: the robot takes about 4 s to drive past a bin at 0.5 m/s, and 15 s gives roughly three times that.
`decay_acceleration`	5.0	Applies only inside the frustum. An obstacle that really has gone still disappears promptly.
`vertical_fov_angle`	0.5236 rad	The VLP-16's 30°, symmetric about horizontal. Declaring the true figure matters: a larger one would put voxels into the frustum that the sensor cannot see, which is the bug.
`vertical_fov_padding`	0.0	Padding *widens* the frustum and so clears more. The package example uses 0.05; it is set to zero here because every metre of padding is a metre of the old failure back.
`horizontal_fov_angle`	6.29 rad	The VLP-16 sees all the way round.
`min_z` / `max_z`	1.0 / 8.0 global, 1.0 / 6.0 local	Near and far range of the frustum (they are radial bounds, not heights, for the 3D-lidar model). 1.0 m is the VLP-16's own minimum usable range, so nothing nearer is ever cleared. The far bound matches `obstacle_range`, so the layer never clears further out than it marks.
`sensor_frame`	velodyne	**Easy to miss and important.** `ramp_detector` republishes the cloud in `base_link`, which is 0.128 m off the ground and 0.369 m ahead of the mast. Without this, STVL would anchor the frustum there instead of at the sensor, and the frustum would not match what the Velodyne can see — which is the entire point of the change.
`voxel_size`	0.025 m	Matched to the cost map resolution, so one voxel is one cell and nothing is coarsened. Raise to 0.05 if CPU becomes a problem.
`track_unknown_space`	false	Matches the cost maps' existing `track_unknown_space: false`; unseen space stays free, as before.
`filter` / `mark_threshold`	"voxel" / 0	The voxel filter collapses the 24 000-point cloud to one point per voxel before insertion, which is most of what keeps this affordable. mark_threshold 0 means a single voxel anywhere in a column marks the cell — the cautious choice.

Marking is otherwise unchanged: same topic (`/velodyne_points_nav`), same 0.2–3.0 m height band, same 8 m / 6 m ranges. The lane and pothole sources carry over as marking-only sources, and they now get a decay timer they did not have before, which is a small bonus for the lane branch.

To undo it: the old `obstacle_layer` block is still in the file, commented out, directly below the new one. Put `"obstacle_layer"` back in the `plugins` list, delete `stvl_layer`, and uncomment the block.

3. Evidence that it works

Two checks were run before handing this back, because a layer swap is a bigger change than a parameter tweak and a typo would only show up on the robot.

1. It loads

A real `nav2_costmap_2d` node was started with the new local-cost-map section and driven through its lifecycle. `configure` returned OK and the log shows all four observation sources built, with no complaint about any parameter:

```
stvl_layer being initialized as SpatioTemporalVoxelLayer!
stvl_layer's global frame is odom.
```

```
  stvl_layer loaded parameters from parameter server.
  stvl_layer created underlying voxel grid.
  Voxel filter activated. / Passthough filter activated. / Voxel filter activated. / Voxel filter
activated.
  stvl_layer initialization complete!
```

2. It keeps the obstacles, on your own data

`claude_tools/replay_costmap.py` replays `/tf_static`, `/tf` and `/velodyne_points_nav` out of **real_bag2** into a live cost map node with the stamps moved to the present, then follows every free-standing obstacle the map holds while the robot is far from it and checks whether it is still there once the robot is close. The same replay was run twice, against the iteration-1 configuration and against this one:

Configuration	Obstacle tracked	Closest approach	Cells far → close	Verdict
Iteration 1 — ObstacleLayer, 2D ray-trace clearing	(+5.57, +0.20)	0.88 m	912 → **0**	**LOST**
Iteration 2 — STVL, frustum + decay	(+5.57, +0.21)	0.89 m	943 → 997	**KEPT**

Same bag, same obstacle, same tracker — the only difference is the cost map configuration. The iteration-1 setup reproduces exactly what was seen on the course: the obstacle is held confidently at 912 cells while the robot is far from it, and is **completely gone** by the time the robot is within a metre. With the frustum rule the same obstacle is still there, slightly larger than before because the robot has had a closer look at it.

This replay harness is the thing to re-run whenever the cost map configuration changes; it is far cheaper than another trip to the course.

4. Everything else, unchanged

Behaviour tree	Untouched. Its last-resort cost-map clears still work: STVL implements `reset()` and is clearable like any other layer.
The rest of Course2027.yaml	Untouched. The speed and yaw envelope, the critics, the goal tolerance, the map sizes and rates from iteration 1 are all as they were.
Worth noting in passing	The iteration-1 rate changes did their job: in these bags `/local_costmap/published_footprint` came out at 9.98 Hz against a 10 Hz target and the global at 5.02 Hz against 5 Hz, where before they were missing their deadlines. Watch this again after the layer swap — STVL costs more than ObstacleLayer, and if the rate slips, `voxel_size` 0.05 is the first thing to try.
One for the bag command	The recorded topic was `/speed_governor_info`; the node publishes `/speed_governor/info` (with a slash). Nothing was captured on it. Not needed for these teleop runs, but it will be wanted for the next driven one.

What to look at next time

Check	Tool	What good looks like
An obstacle the robot drives up to stays in the map	`obstacle_persistence.py`	every tracked obstacle reads KEPT; nothing vanishes inside 2 m
Marks still disappear when they should	`erasure_range.py`	a mark with no returns behind it is gone within about 15 s, not never

Check	Tool	What good looks like
The corridor does not close in, as it did in iteration 1	`corridor_trend.py`	free width across the robot stays above 1.4 m in the 2.13 m hallway
The controller still makes its rate	`bag_summary.py`	`/cmd_vel_raw` near 10 Hz, footprints at 10 and 5 Hz

New tools this iteration, all in `claude_tools/`: `obstacle_persistence.py` (does an obstacle survive the approach), `cloud_vs_cloud.py` (which stage lost it — sensor, `ramp_detector`, or cost map), `erasure_range.py` (at what range the map erases things), `sensor_geometry.py` (the blind-cone table above), `obstacle_heights.py` (how tall the course obstacles actually are), `scene_frames.py` (cost map and LiDAR side by side over time) and `replay_costmap.py` (the A/B harness).